

Árboles de decisión

Desarrollo de Aplicaciones Avanzadas de Ciencias Computacionales
Inteligencia Artificial
(TC3002B.M2)

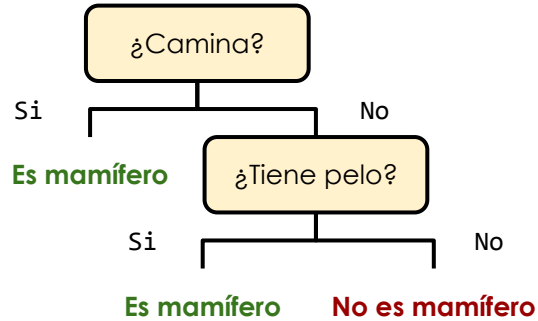
José Carlos Ortiz Bayliss



Árboles de decisión

- En aprendizaje automático, un árbol de decisión es una estructura (en forma de árbol) en la que cada nodo representa una variable o característica, y las ramas representan reglas de decisión. Las hojas del árbol representan los resultados.
- Los datos se dividen de forma recursiva, buscando maximizar las diferencias observables en las características y su impacto en el proceso de clasificación.
- Los árboles de decisión son usados tanto para clasificación como para regresión, aunque se volvieron populares en problemas de clasificación.
- Al igual que las reglas de decisión, trabajan con variables independientes cuantitativas y cualitativas, aunque su desempeño es mejor cuando se usan variables cualitativas.

¿Cómo podemos clasificar mediante reglas de decisión?

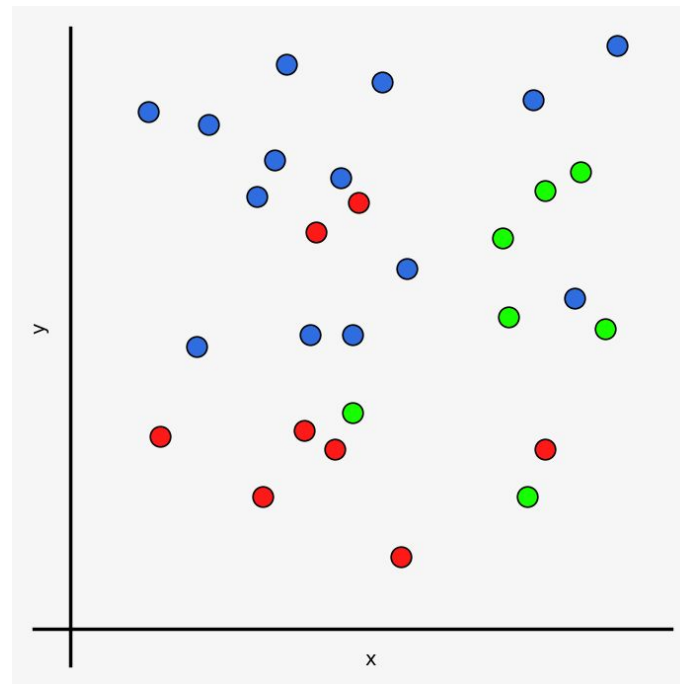


¿Cómo se construye un árbol de decisión?

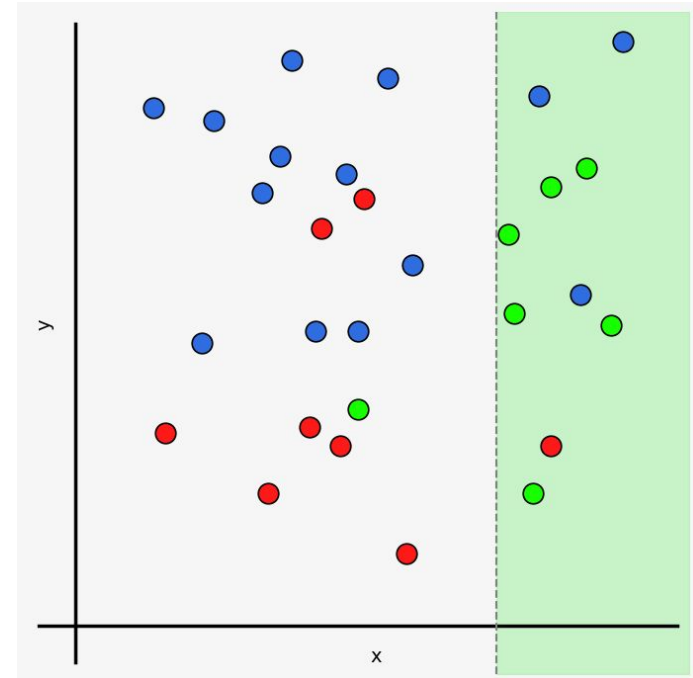
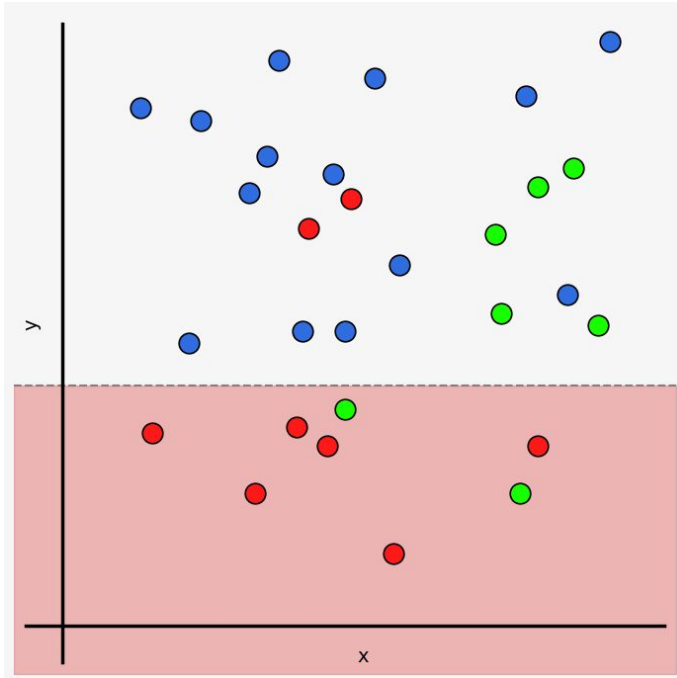
Un árbol de decisión se construye de manera recursiva y codiciosa:

- I. Se empieza con todos los datos en un solo nodo.
- II. Se buscan posibles cortes sobre cada variable y se evalúa cuál separa mejor las clases (usando una medida de impureza como Gini o entropía).
- III. Se elige el mejor corte y se divide el conjunto en dos subconjuntos.
- IV. Se repite el mismo proceso en cada subconjunto.
- V. El proceso se detiene cuando los datos ya no se pueden mejorar (por ejemplo, cuando el nodo contiene sólo una clase o ya no hay cortes útiles).

El resultado es una estructura de reglas simples encadenadas que, en conjunto, permiten clasificar nuevos datos.



Posibles cortes (por intuición)



¿Cómo se definen los cortes?

Para variables cuantitativas:

- Para cada variable x_i , se ordenan sus valores.
- Se crean umbrales (valores intermedios entre valores consecutivos).
- Para cada umbral t , se evalúa la separación resultante ($x_i \leq t$ vs. $x_i > t$) y se escoge el mejor.

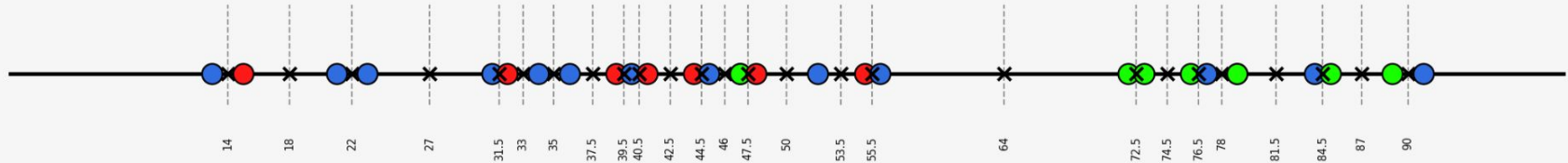
Para variables cualitativas:

- Para cada variable x_i , se evalúa la separación resultante de cada categoría S_i ($x_i = S_i$ vs. $x_i \neq S_i$) y se escoge el mejor*.

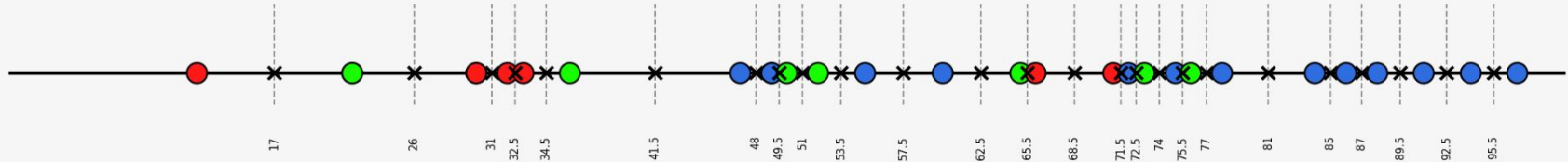
* Si el número de categorías es pequeño, se evalúan todas las categorías. Si no, se usan heurísticas para determinar cuales categorías evaluar.

Valores y umbrales

Variable x



Variable y



El índice Gini

- El índice Gini mide qué tan "mezcladas" están las clases en un nodo.
- Para un nodo con k clases, donde p_k es la proporción de la clase k en el nodo, el índice Gini se calcula como:

$$Gini = 1 - \sum_{k=1}^K p_k^2$$

- Un índice Gini bajo sugiere una mejor distribución de clases en la partición.

Evaluación de las particiones

Se calcula el Gini de los nodos hijos, donde n_L y n_R son el número de ejemplos contenidos en los grupos formados por la partición de datos, y n es el total de datos ($n_L + n_R$):

$$Gini_{split} = \frac{n_L}{n} Gini_L + \frac{n_R}{n} Gini_R$$

Un valor de Gini split bajo representa una mejor partición de los datos.

Evaluación de particiones ($x = 64$)

Partición izquierda

$x \leq 64$

● 7 ● 1 ● 11

Partición derecha

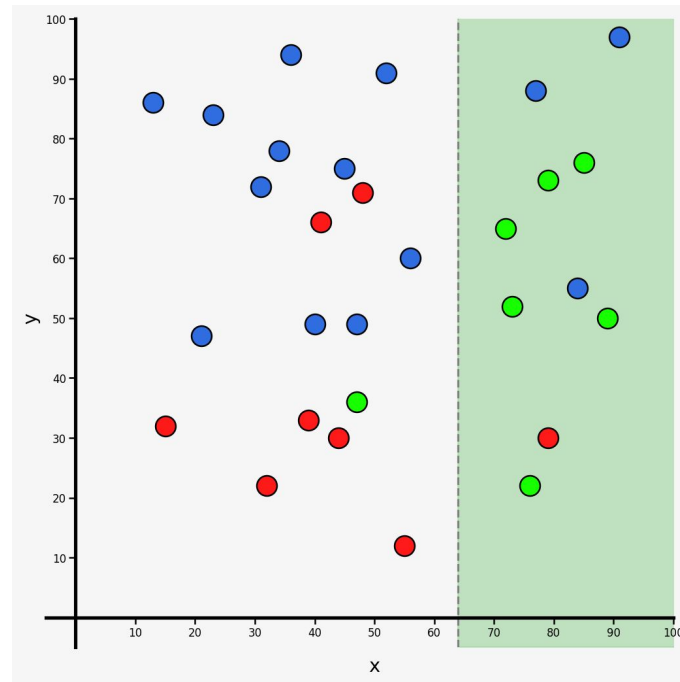
$x > 64$

● 1 ● 6 ● 3

$$\begin{aligned} Gini_L &= 1 - \left(\left(\frac{7}{19} \right)^2 + \left(\frac{1}{19} \right)^2 + \left(\frac{11}{19} \right)^2 \right) \\ &= 1 - (0.1357 + 0.0028 + 0.3352) \\ &= 1 - 0.4737 = 0.5263 \end{aligned}$$

$$\begin{aligned} Gini_R &= 1 - \left(\left(\frac{7}{19} \right)^2 + \left(\frac{1}{19} \right)^2 + \left(\frac{11}{19} \right)^2 \right) \\ &= 1 - (0.0100 + 0.3600 + 0.0900) \\ &= 1 - 0.4600 = 0.5400 \end{aligned}$$

$$\begin{aligned} Gini_{split} &= \frac{19}{29} (0.5263) + \frac{10}{29} (0.5400) \\ &= 0.3448 + 0.1862 = 0.5310 \end{aligned}$$



Evaluación de particiones ($y = 41.5$)

Partición inferior

$$y \leq 41.5$$

● 6 ● 2 ● 0

Partición superior

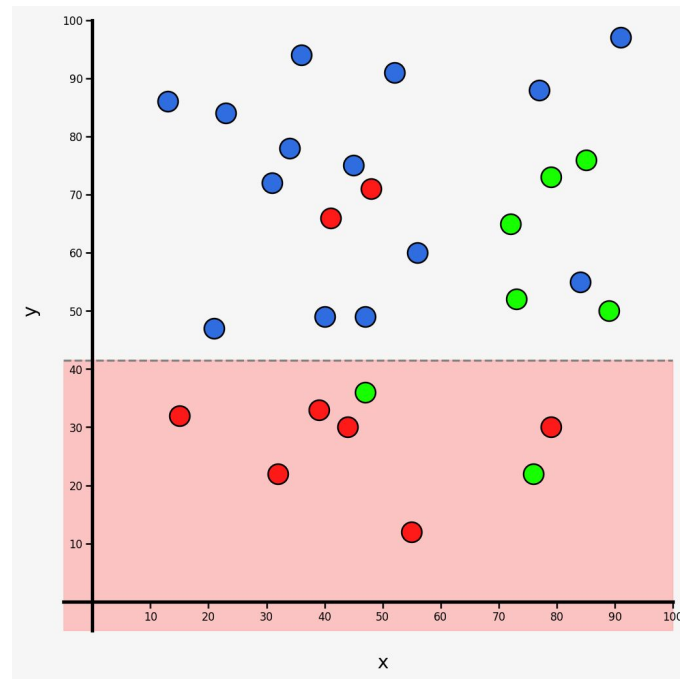
$$y > 41.5$$

● 2 ● 5 ● 14

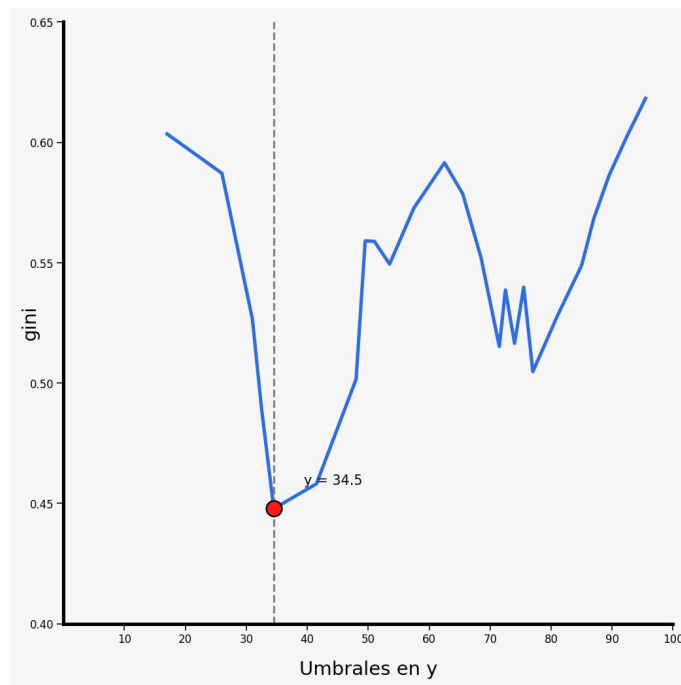
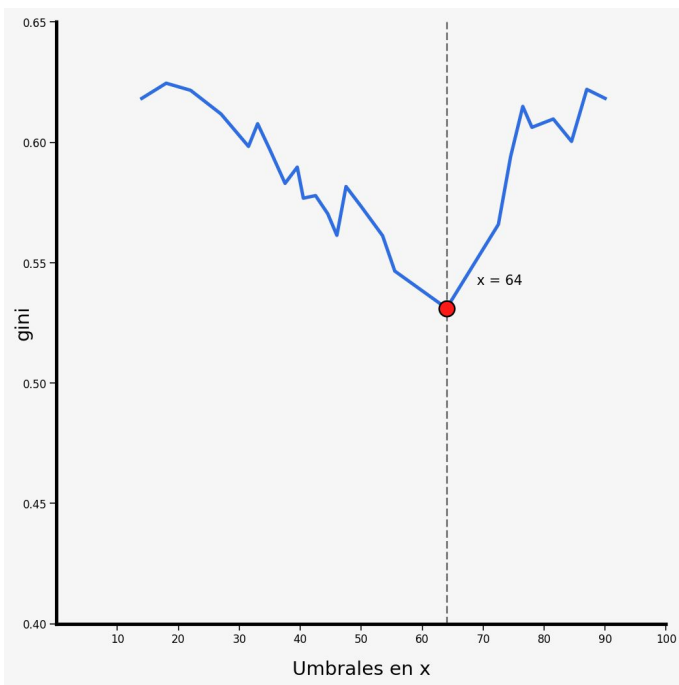
$$\begin{aligned} Gini_L &= 1 - \left(\left(\frac{6}{8} \right)^2 + \left(\frac{2}{8} \right)^2 + \left(\frac{0}{8} \right)^2 \right) \\ &= 1 - (0.5625 + 0.0625 + 0.0) \\ &= 1 - 0.6250 = 0.3750 \end{aligned}$$

$$\begin{aligned} Gini_R &= 1 - \left(\left(\frac{2}{21} \right)^2 + \left(\frac{5}{21} \right)^2 + \left(\frac{14}{21} \right)^2 \right) \\ &= 1 - (0.0091 + 0.0567 + 0.4444) \\ &= 1 - 0.5102 = 0.4898 \end{aligned}$$

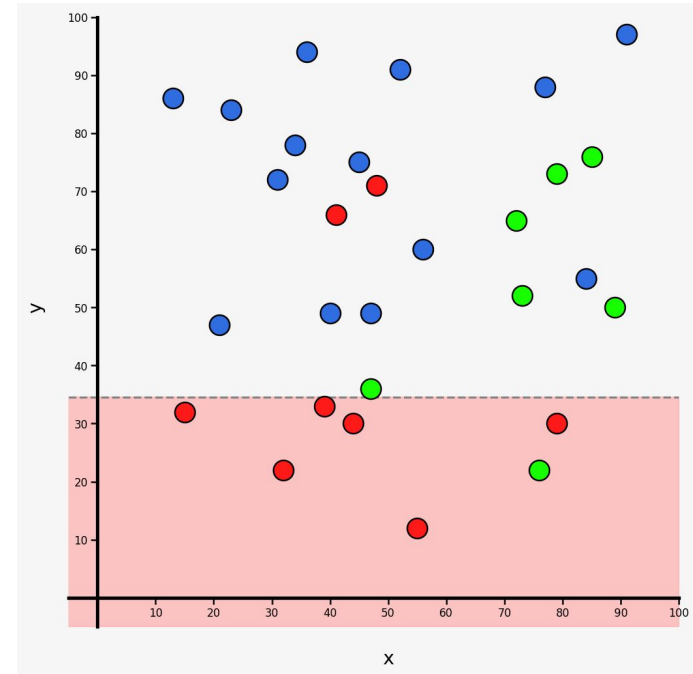
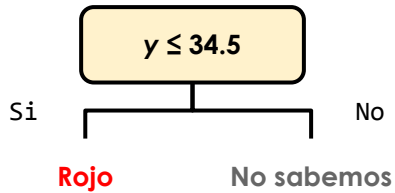
$$\begin{aligned} Gini_{split} &= \frac{8}{29}(0.3750) + \frac{21}{29}(0.4898) \\ &= 0.1034 + 0.3547 = 0.4581 \end{aligned}$$



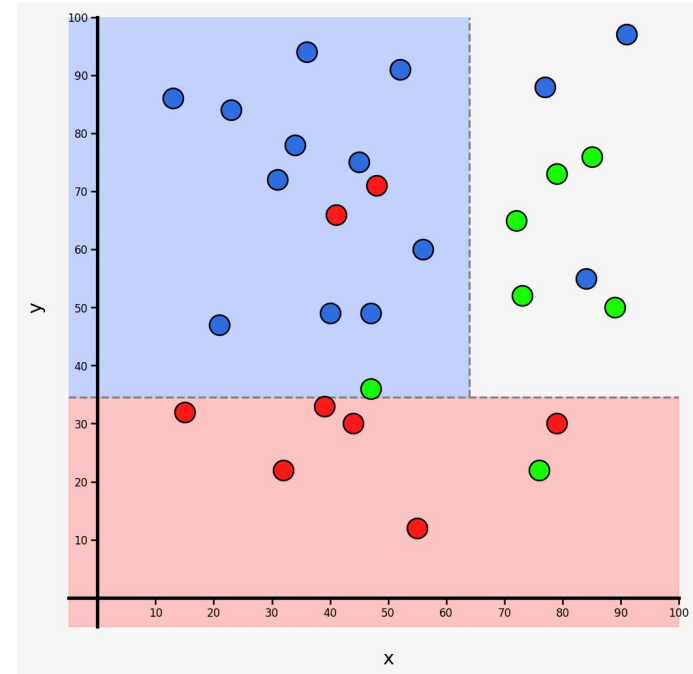
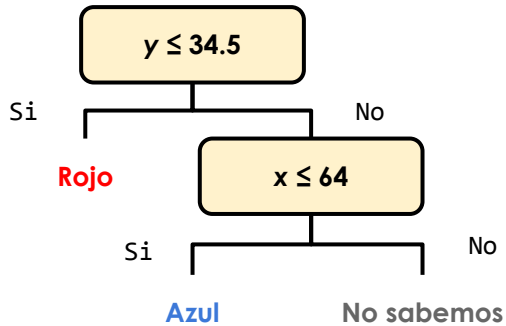
Evaluación de las particiones por cada variable



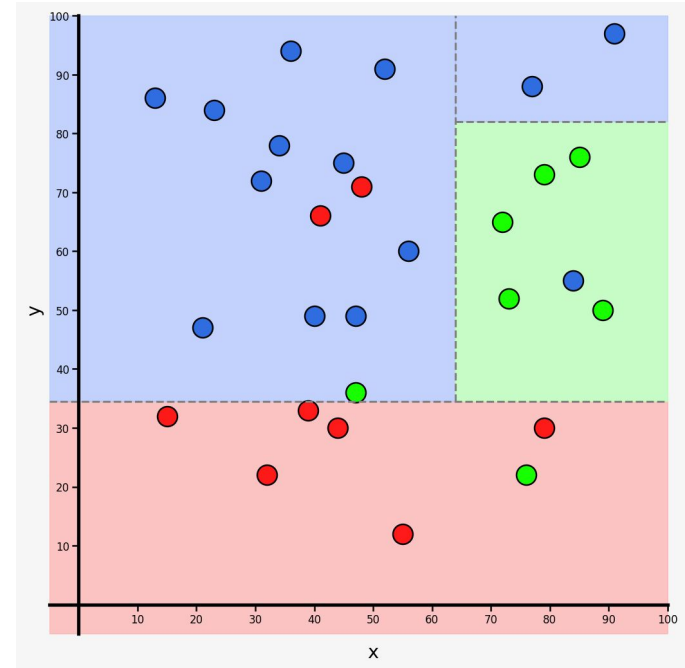
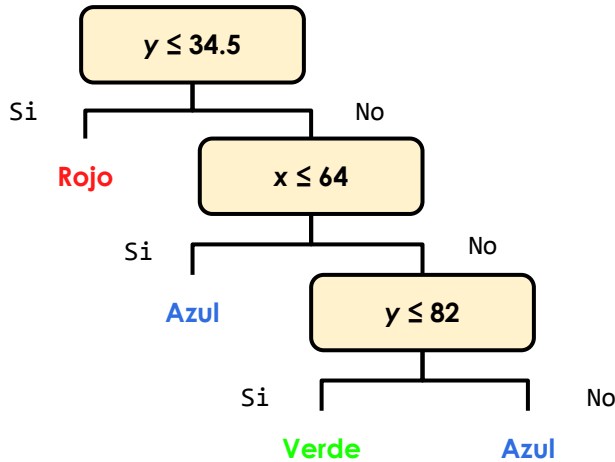
¿Cómo se construye un árbol de decisión?



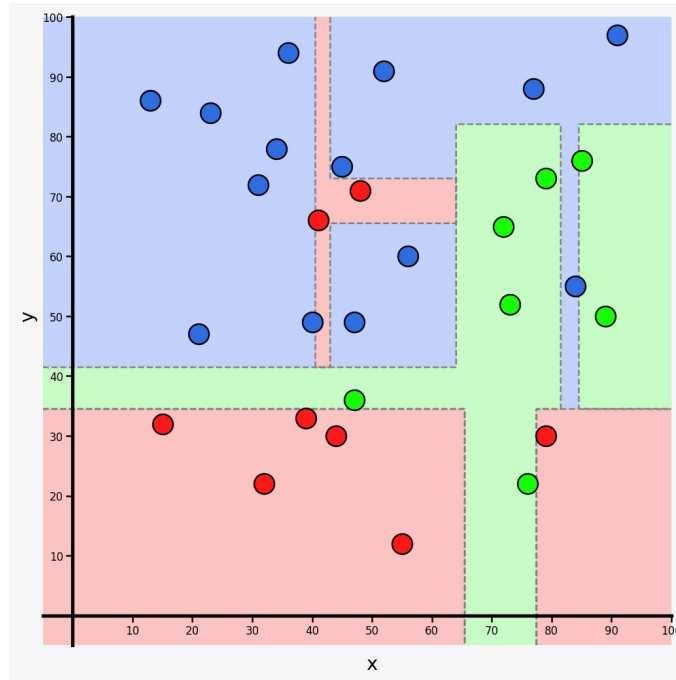
¿Cómo se construye un árbol de decisión?



¿Cómo se construye un árbol de decisión?



El riesgo de sobreentrenamiento



La importancia de entender cómo se toman las decisiones

- La estructura en forma de árbol por donde fluye la información no sólo es de utilidad para la máquina.
- El árbol resultante puede interpretarse por los humanos, de forma que puede entenderse por qué funciona o no (para la mayoría de los casos).
- Esto contrasta con otros métodos de aprendizaje automático, como son las redes neuronales y las máquinas de vectores de soporte, donde las decisiones son difíciles de interpretar.
- Aunque existen diferentes algoritmos para generar árboles de decisión, las ideas sobre la forma de generar el árbol y su interpretación aplican para cualquiera de ellos.

Generación del modelo de clasificación con árboles de decisión

```
# Importamos los módulos necesarios
import numpy as np
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import confusion_matrix
from sklearn.model_selection import train_test_split

# Leemos los datos que usaremos
iris = pd.read_csv("iris.csv")

# Separamos las variables independientes de la dependiente
X = iris.drop("Species", axis = 1)
y = iris["Species"]

# Dividimos los datos en entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size = 0.60, random_state = 42)
```

Generación del modelo de clasificación con árboles de decisión

```
# Creamos un modelo de clasificación usando un árbol de decisión
model = DecisionTreeClassifier(random_state = 42)

# Entrenamos el modelo usando los datos de entrenamiento
model.fit(X_train, y_train)

# Realizamos la predicción usando los datos de prueba
prediction = model.predict(X_test)

# Generamos la matriz de confusión
confusion_matrix(y_test, prediction)
```

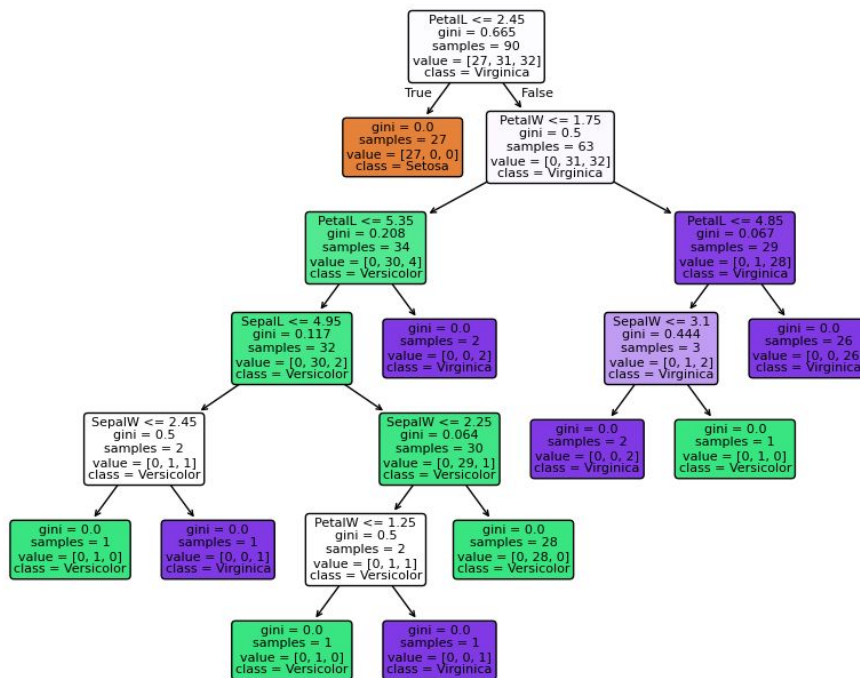
Evaluación del desempeño

		Predicción		
		Versicolor	Setosa	Virginica
Realidad	Versicolor	23	0	0
	Setosa	0	18	1
	Virginica		1	17

Visualización del árbol de decisión generado

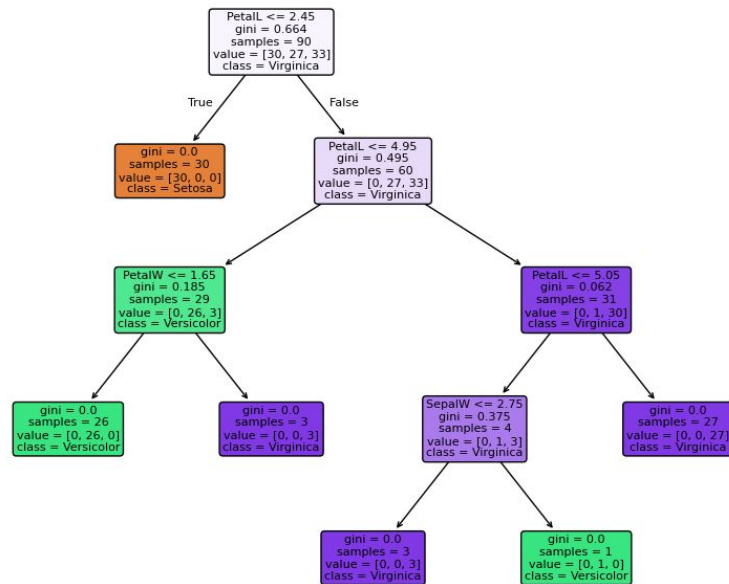
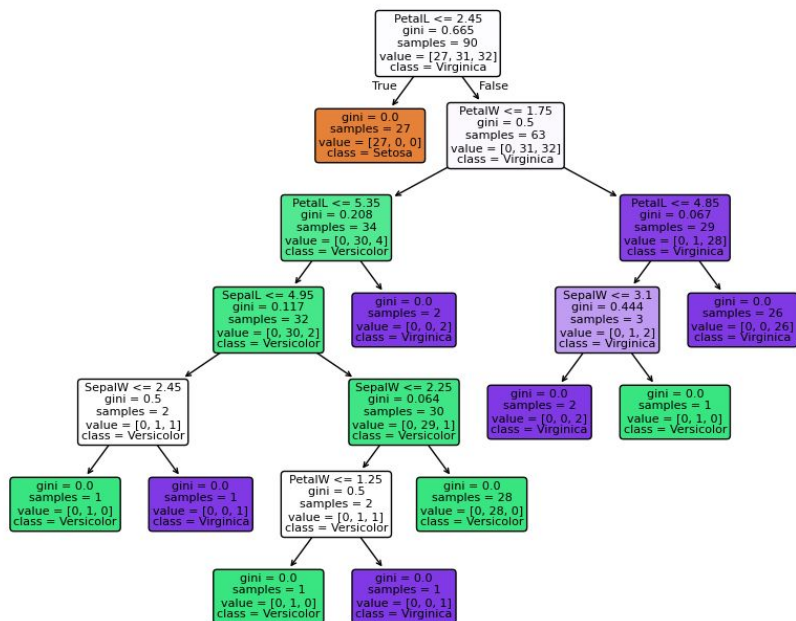
```
# Importamos los módulos necesarios
import matplotlib.pyplot as plt
from sklearn.tree import plot_tree

# Visualizamos el árbol de decisión generado
plt.figure(figsize=(10, 8))
plot_tree(
    model,
    feature_names = X.columns,
    class_names=[str(c) for c in model.classes_],
    filled = True,
    rounded = True,
    impurity = True,
    proportion = False,
    fontsize = 8
)
```



Sensibilidad a los datos de entrenamiento

Los cambios en el conjunto de entrenamiento pueden producir árboles muy diferentes (aún con desempeños similares).



Robustez ante diferentes escalas de las variables

A diferencia de métodos como k vecinos más cercanos o máquinas de vectores de soporte, los árboles de decisión pueden trabajar con variables cuantitativas en rangos diferentes.

```
# Importamos los módulos necesarios
import numpy as np
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import confusion_matrix
from sklearn.model_selection import train_test_split

# Leemos los datos que usaremos
iris = pd.read_csv( "distance.csv" )

# Separamos las variables independientes de la dependiente
X = iris.drop( "Output", axis = 1)
y = iris[ "Output" ]
```

Robustez ante diferentes escalas de las variables

```
# Dividimos los datos en entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size = 0.60, random_state = 54321)

# Creamos un modelo de clasificación usando un árbol de decisión
model = DecisionTreeClassifier(random_state = 42)

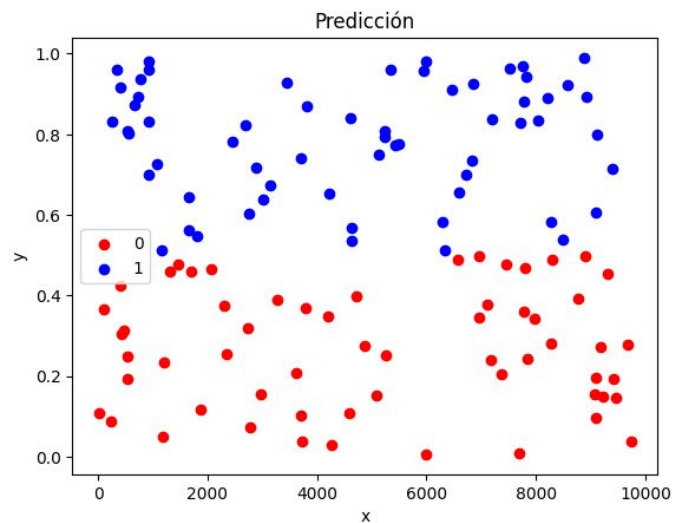
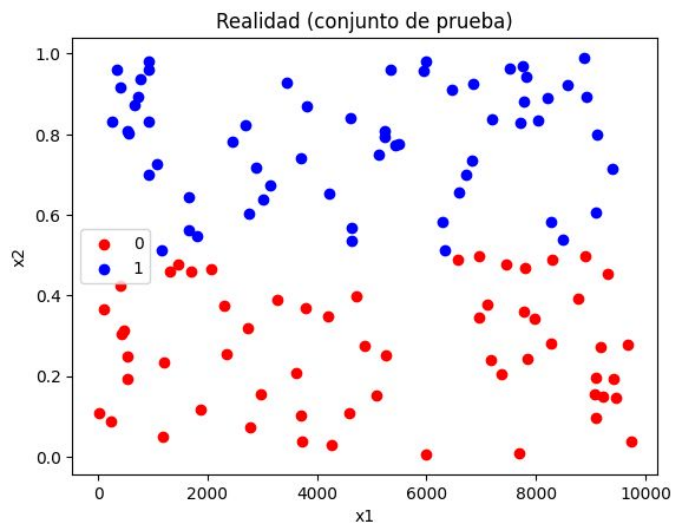
# Entrenamos el modelo usando los datos de entrenamiento
model.fit(X_train, y_train)

# Realizamos la predicción usando los datos de prueba
prediction = model.predict(X_test)

# Generamos la matriz de confusión
confusion_matrix(y_test, prediction)
```

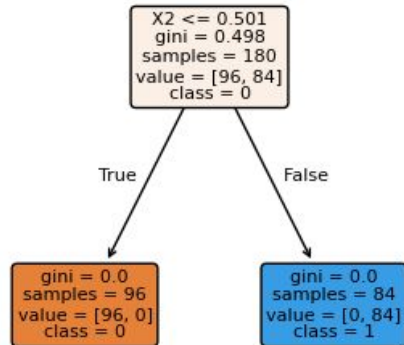

Evaluación del desempeño

		Predicción	
		close	far
Realidad	close	59	0
	far	0	61



Visualización del árbol de decisión generado

```
# Visualizamos el árbol de decisión generado
plt.figure(figsize=(4, 4))
plot_tree(
    model,
    feature_names = X.columns,
    class_names=[str(c) for c in model.classes_],
    filled = True,
    rounded = True,
    impurity = True,
    proportion = False,
    fontsize = 8
)
```



Conclusiones

- Si el árbol generado tiene un tamaño "razonable", sus decisiones son fáciles de entender.
- El proceso no requiere escalar los datos.
- Descarta variables irrelevantes de forma automática y puede trabajar tanto con variables cuantitativas como cualitativas.
- Pequeños cambios en el conjunto de entrenamiento pueden producir grandes cambios en la estructura del árbol.
- Lamentablemente, tiende a subentrenar y sobreentrenar.

Recursos adicionales

- <https://sklearn.com/>
- <https://scikit-learn.org/>
- <https://developers.google.com/machine-learning/crash-course>